

Reflective Report: Ezypick

Peter Wanchai · Advanced iOS Development · Assessment 2

What the domain taught me

I started Assessment 2 assuming I would build what I pitched in Assessment 1: a swipe deck of cuisine and vibe cards ending on one restaurant. Two things I found in the domain changed that.

The first was about the questions. I had assumed "Feeling Italian?" was doing useful work. It is not. Of the nine places a live search found around me, one was Thai, so answering no to one cuisine rules out almost nothing, while "Got time to sit down properly?" splits the same set roughly in half. I had chosen the deck because it looked good as an interaction rather than because it decided anything. So the app now generates questions from the restaurants still in the running instead of a list written in advance.

The second was about dietary requirements, and it cost me the hardest rule I wrote. I assumed the data would be there. It is not. Gluten-free has no field anywhere in the API. Halal is a category almost nobody publishes: one venue in the inner city, so it came back empty. Vegetarian is the subtle one: the flag exists, and it is almost never a no. Two venues in fifty-seven say false, and absent does not mean no, so the veto ruled out two places while looking like a safety check. I built the rule anyway. Empty, vacuous or absent: none of the three protected anyone, so the rule went rather than got hedged, and the app now filters on the three things a phone can actually know.

Why these use cases

I ended up with five use cases, and four of them own a rule that would cause a real problem if it broke.

`ShortlistRestaurantsUseCase` holds the hard limits: budget, walking distance, whether the place is open right now, and anything already turned down. It runs before a single question is asked, so nothing unaffordable or shut ever reaches the part of the app that is less careful.

`AskNextQuestionsUseCase` holds the rules about asking: at most five questions, stop as soon as three restaurants remain, never repeat, and never ask anything the remaining restaurants would all answer the same way. An app built to save someone from choosing cannot then put them through a quiz. In testing it usually stops after one or two questions.

The other two are smaller. `SaveDiningPreferencesUseCase` stops someone saving a profile that could never return a result, and `AnswerQuestionUseCase` refuses an answer that would leave them with nowhere to eat.

Putting the language model behind a protocol mattered more than I expected. The model writes the questions, but the use case checks each one against the real candidates and discards it if it would not narrow anything. With no network a deterministic generator takes over; with no key it stops at the shortlist and says so. The model phrases; it never decides.

Where a person can go wrong

The moment I worried about most is the profile screen. Someone sets their walking limit to five minutes on a rainy day and forgets. Every lunch after that is quietly starved of options, and they would blame the app rather than the setting.

My answer was to make failure name its own cause. The filtering step counts what each limit excluded, so when nothing survives, the app can say "the places that fit are all further than you said you'd walk" and name the fix, instead of showing "no results". The count is four integers, and it turns a dead end into something the person can act on: for a budget, one tap.

The same thinking shaped the profile screen itself. It has two controls, a budget and a walk. What you feel like eating today is not on it, because that is what the questions are for: one is never traded away, the other is. On one screen they would look the same, and the person would have no way of knowing which is which.

What I would do next

Live restaurant data is in: one new type behind `RestaurantRepository`, with no rule and no screen changed, which is the best evidence I have that the port earned its place. Cost is still real, because Places charges per field for reviews and atmosphere data, so a paying version has to choose which restaurants are worth fetching in detail.

Dietary, properly, is still ahead. It needs a source that records what kitchens actually do, and no mapping platform is one, so it is a data problem before it is an app problem.

After that, group lunches, which I cut from this version. I would rather ship one honest single-user version than a half-working group one.